

Chapitre 5

Configuration Réseau

Introduction : Le paradigme réseau moderne

L'évolution de l'administration réseau sous Linux :

- L'utilisation d'outils historiques isolés (ifconfig) est révolue.
- Sous Ubuntu, Netplan est devenu la méthode recommandée : l'administrateur décrit *l'état final désiré* de l'infrastructure sous forme de fichiers **YAML**. Le système calcule et applique les changements nécessaires.

Important

La distribution **Debian** utilise encore le fichier `/etc/network/interfaces` pour la configuration du réseau (idem pour **Kali Linux**).

RedHat utilise d'autres fichiers de configuration.

Nomenclature prédictive

Les interfaces ne s'appellent plus `eth0`, `eth1`, etc.. Les systèmes actuels utilisent des noms basés sur la topologie matérielle, évitant que les noms changent au redémarrage.

Décryptage du nom (ex : `enp3s0`) :

- **en** : Type de média (**en** pour Ethernet, **wl** pour WLAN).
- **p3** : Numéro du bus PCI (bus 3).
- **s0** : Numéro de l'emplacement/slot (slot 0).

Rigueur administrative

En production, ne tentez jamais de forcer le retour à l'ancienne nomenclature (`eth0`) sur des serveurs modernes.

La commande ip : Remplaçante de ifconfig

La commande `ip` fait partie du paquet **iproute2**. Elle est le standard moderne sous Linux, remplaçant les anciens outils comme `ifconfig` et `route` (paquet `net-tools`) considérés comme obsolètes.

Afficher les adresses IP de toutes les interfaces :

```
ip address show  
# ou la version courte  
ip a
```

Éléments clés de la sortie standard :

- **lo** : Interface de bouclage (loopback, adresse 127.0.0.1).
- **enp3s0** : Nom de l'interface réseau physique.
- **LOWER_UP** : Indique que le lien physique est actif (câble branché).
- **inet** / **inet6** : Indique respectivement les adresses IPv4 et IPv6.

Filtrer l'affichage des adresses IP

Afficher les informations pour une interface spécifique :

```
ip address show dev enp3s0
```

N'afficher que les interfaces actives (UP) :

```
ip link ls up
```

N'afficher que les adresses IPv4 :

```
ip -4 address show
```

La commande ip : Table de routage

Afficher la table de routage :

```
ip route show  
# ou la version courte  
ip r
```

Exemple de sortie :

```
default via 192.168.1.254 dev enp3s0 proto dhcp metric 100  
192.168.1.0/24 dev enp3s0 proto kernel scope link src 192.168.1.50
```

- **default via 192.168.1.254** : Définit l'adresse du routeur (la porte de sortie).
- **dev enp3s0** : Indique par quelle carte réseau les paquets vont sortir.
- **src 192.168.1.50** : Indique l'adresse IP source de la machine sur ce réseau.

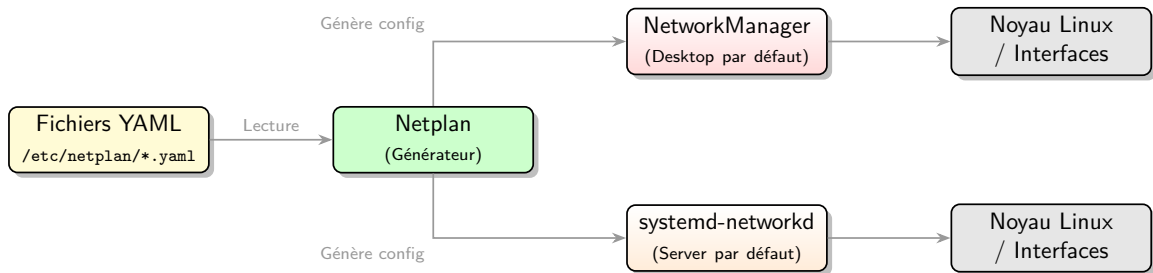
Introduction à Netplan

Depuis la version 17.10, Ubuntu utilise **Netplan** comme utilitaire de gestion de réseau par défaut, remplaçant le classique `/etc/network/interfaces`.

Principes de base :

- Les configurations sont écrites en **YAML** dans `/etc/netplan/`.
- Netplan lit ces fichiers au démarrage et génère la configuration pour le backend approprié.
- **Ubuntu Server** utilise généralement `systemd-networkd` comme backend.
- **Ubuntu Desktop (Mint)** utilise `NetworkManager` (permettant la gestion via l'interface graphique).

Architecture de configuration



Netplan : Comprendre le *renderer*

Netplan n'est qu'un générateur de configuration. Le vrai travail est fait par un **backend**.

Serveur

renderer: networkd

- Léger, sans interface graphique
- Géré par systemd-networkd

Bureau

renderer: NetworkManager

- Permet la gestion GUI et nmcli
- Idéal pour les environnements dynamiques (Wi-Fi, VPN)

Règle d'or

Sur un serveur en production, utilisez networkd. N'installez jamais NetworkManager.

Configuration Dynamique (DHCP)

Dans un environnement client, l'adresse IP est souvent attribuée par un serveur DHCP.

Exemple de fichier Netplan (/etc/netplan/50-cloud-init.yaml) :

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      dhcp4: true
      dhcp6: false
```

- `renderer` : Spécifie le backend .
- `dhcp4: true` : Active le client DHCP pour l'IPv4.
- `dhcp6: false` : Désactive le client DHCP pour l'IPv6.

Note : *L'indentation (espaces, pas de tabulations) est stricte en YAML.*

Configuration Statique

Pour un serveur, il est critique d'avoir une adresse IP fixe pour garantir l'accessibilité des services (Web, DNS, Base de données).

Exemple avec IP Statique :

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      addresses:
        - 192.168.1.100/24
      routes:
        - to: default
          via: 192.168.1.254
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Risque majeur

Les fichiers YAML peuvent contenir des mots de passe. Ne jamais les laisser lisibles par tous.

Permissions obligatoires

```
# Le fichier doit être lisible uniquement par root
# Vérification
ls -l /etc/netplan/
# -rw----- 1 root root ... 50-cloud-init.yaml

# sinon
sudo chmod 0600 /etc/netplan/50-cloud-init.yaml
sudo chown root:root /etc/netplan/50-cloud-init.yaml
```

Configuration IPv6

L'adressage IPv4 exclusif est obsolète. Un administrateur système doit configurer le double pile (Dual-Stack) IPv4/IPv6.

```
network:
  version: 2
  ethernets:
    enp3s0:
      addresses:
        - 192.168.1.100/24
        - "2001:db8:1::100/64" # Les guillemets évitent les conflits YAML
      routes:
        - to: default
          via: 192.168.1.254
        - to: default
          via: "2001:db8:1::1"
```

Appliquer la configuration Netplan

Une fois le fichier YAML modifié, il faut vérifier, tester et appliquer la configuration.

Vérifier la syntaxe YAML sans appliquer

```
sudo netplan generate
```

En cas d'erreur, corriger avant de casser le réseau.

Tester la configuration (Recommandé)

```
sudo netplan try
```

Cette commande applique la configuration temporairement. Si vous perdez la connexion, elle annule les changements après 120 secondes.

Appliquer de manière permanente

```
sudo netplan apply
```

Règle d'or : Ne jamais perdre la main

En production

Appliquer une mauvaise configuration réseau sur un serveur distant = perte définitive de l'accès (besoin d'intervention physique).

Procédure de sécurité (à exécuter avant netplan apply)

- 1 Ouvrir **deux** sessions SSH simultanées
- 2 Planifier un reboot de secours (si la config plante, le serveur revient à l'ancienne config) :
`sudo shutdown -r +5 "Redémarrage de secours dans 5min"`
- 3 Tester la config :
`sudo netplan try`
- 4 Si OK, annuler le redémarrage :
`sudo shutdown -c`

Outils de diagnostic réseau

Commandes essentielles pour vérifier que la configuration a bien été appliquée :

- **Vérifier l'adresse IP :**

```
ip a
```

- **Vérifier la table de routage :**

```
ip r
```

- **Tester la connectivité (DNS et accès externe) :**

```
ping -c 4 google.fr
```


La commande ping ne valide que la couche 3 (Réseau). Pour valider le bon fonctionnement des services, il faut inspecter les sockets et utiliser des requêtes ciblées.

- **Vérifier les ports en écoute :**

```
ss -tulpen  
# t: TCP, u: UDP, l: listening, p: process, e: extended, n: numeric
```

Permet de s'assurer qu'un service comme SSH écoute bien sur le port 22.

- **Tester la résolution DNS :**

```
dig +short A www.ump.ma  
dig +short AAAA www.ump.ma # Pour l'IPv6
```

Commandes supplémentaires indispensables

- `arping -c3 -I enp3s0 10.0.2.2`
teste la passerelle au niveau MAC (couche 2). **arping** n'est pas installé par défaut
`sudo apt install arping`
- `traceroute -n 8.8.8.8`
montre chaque saut jusqu'à la destination. **traceroute** n'est pas installé par défaut
`sudo apt install inetutils-traceroute`
- `ss -t state established`
connexions TCP établies
- `ip neigh`
table ARP (voisins)

Résolution de noms en local : /etc/hosts

Avant d'interroger un serveur DNS, Linux consulte **d'abord** le fichier /etc/hosts.

Ordre de résolution (configuré dans /etc/nsswitch.conf)

hosts: files dns

- files = /etc/hosts
- dns = serveurs configurés (via Netplan ou resolvectl)

Fichier /etc/hosts

```
127.0.0.1    localhost
192.168.1.100  info.local
```

Permet de joindre info.local sans DNS.

Important

/etc/hosts est indispensable en cas de DNS en panne ou pour les petits réseaux.

Résolution DNS locale (systemd-resolved)

Sous Ubuntu, le fichier historique `/etc/resolv.conf` n'est plus géré manuellement. C'est un lien symbolique pointant vers le service `systemd-resolved` (qui écoute sur `127.0.0.53`).

Diagnostiquer et interroger le DNS proprement :

```
# Vérifier les serveurs DNS actuellement utilisés par les interfaces
resolvectl status

# Faire une requête DNS spécifique via ce service
resolvectl query www.ump.ma
```

Attention

Il faut noter que la modification de `/etc/resolv.conf` à la main sous Ubuntu est une erreur, car le fichier sera écrasé par **systemd-resolved** au prochain redémarrage.

Capture et analyse de paquets

tcpdump : Outil de dernier recours

Capturer le trafic ARP et ICMP (diagnostic base)

```
sudo tcpdump -i enp0s3 arp or icmp -n
```

Vérifier si SSH arrive bien sur l'interface

```
sudo tcpdump -i any port 22 -n -c 10
```

Sauvegarde pour analyse Wireshark ultérieure

```
sudo tcpdump -i enp0s3 -w /tmp/capture.pcap
```

Précaution légale

Ne jamais capturer de trafic contenant des données personnelles sans autorisation.

Alias IP (Adresses Secondaires)

Cas d'usage

Héberger plusieurs sites sur des IPs distinctes.

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s9:
      addresses:
        - 192.168.1.10/24      # IP principale
        - 192.168.1.11/24     # Alias 1
        - 192.168.1.12/24     # Alias 2
```

Attention

Les alias ne créent pas d'interfaces physiques `enp3s0:0` (style ancien `ifconfig`), mais sont des labels sur la même interface.

Accès à Distance SSH (Secure Shell)

Introduction à SSH

SSH (Secure Shell) est le protocole standard pour l'administration sécurisée de machines à distance. Il remplace les anciens protocoles en clair comme Telnet.

Concepts clés :

- Chiffrement des communications de bout en bout.
- Fonctionne sur le **port TCP 22** par défaut.
- Architecture Client / Serveur.

Installation sous Ubuntu :

```
# Installer le serveur SSH (sur la machine à contrôler)
sudo apt update && sudo apt install openssh-server

# Le client est généralement installé par défaut (openssh-client)
sudo apt install openssh-client
```

Note : *Sous Ubuntu Server, OpenSSH est proposé lors de l'installation de l'OS.*

Utilisation du Client SSH

La commande `ssh` permet de se connecter au serveur distant depuis le terminal du client.

Syntaxe de base :

```
ssh utilisateur@adresse_ip  
# Exemple :  
ssh admin@192.168.1.100
```

Options courantes :

- `-p <port>` : Spécifier un port non standard (ex : `ssh -p 2222 user@ip`).
- `-X` : Activer le transfert d'affichage graphique (X11 Forwarding).

Avertissement

Lors de la première connexion, le client demande d'accepter l'empreinte cryptographique (fingerprint) du serveur pour contrer les attaques.

Authentification par Clés (Cryptographie asymétrique)

Pour plus de sécurité et de confort, utilisez des paires de clés (publique/privée) plutôt que des mots de passe :

❶ Générer la paire de clés (sur le client) :

```
ssh-keygen -t ed25519 -C "m2i@192.168.56.102"
```

L'algorithme Ed25519 est fortement recommandé pour sa sécurité robuste et ses performances.

❷ Copier la clé publique sur le serveur :

```
ssh-copy-id m2i@192.168.56.102
```

Cette commande ajoute la clé publique au fichier « ~/.ssh/authorized_keys » de l'utilisateur distant.

Configuration du Serveur SSH

Le comportement du serveur se configure dans le fichier « /etc/ssh/sshd_config ». Il est crucial de le sécuriser sur un serveur de production.

Bonnes pratiques de sécurité :

```
# Interdire la connexion directe en tant que superutilisateur (root)
PermitRootLogin no

# Changer le port d'écoute pour limiter les attaques automatisées
# Port 2222

# Désactiver l'authentification par mot de passe (si clés actives)
# PasswordAuthentication no
```

Application des changements :

```
sudo systemctl reload ssh
```

Permissions strictes SSH

Règle absolue

Si SSH bloque l'accès en cas de permissions trop permissives, c'est pour protéger l'utilisateur.

Permissions obligatoires côté serveur

```
# Sur le serveur distant (~/.ssh)
chmod 700 ~/.ssh                # rwx-----
chmod 600 ~/.ssh/authorized_keys # rw-----
chmod 644 ~/.ssh/known_hosts    # rw-r--r-- (optionnel)

# Le répertoire home ne doit pas être writable par groupe/autres
chmod 755 $HOME # ou 700, mais jamais 777
```

Vérification des logs si échec

```
cat /var/log/auth.log | grep sshd
```

Sécurité : Pare-feu avec UFW (Uncomplicated Firewall)

Introduction à UFW

Sous Linux, le filtrage réseau historique est géré par **Netfilter** (via iptables ou nftables). Ces outils étant complexes, Ubuntu propose **UFW** pour simplifier l'administration.

Principes de base d'UFW :

- C'est une interface simplifiée (frontend) pour iptables/nftables.
- Par défaut, il est **désactivé**.
- Sa politique par défaut (une fois activé) est : **Refuser tout le trafic entrant** et **Autoriser tout le trafic sortant**.

Vérifier l'état du pare-feu :

```
sudo ufw status  
# Pour plus de détails :  
sudo ufw status verbose
```

Règles de base et Activation

Attention : Piège classique !

Si vous administrez un serveur à distance via SSH, vous **devez absolument** autoriser le port SSH avant d'activer UFW, sous peine d'être immédiatement déconnecté.

① Autoriser le trafic (ex : SSH et HTTP) :

```
# Via le nom du service
```

```
sudo ufw allow ssh
```

```
sudo ufw allow http
```

```
# Via le numéro de port (pratique si port non standard)
```

```
sudo ufw allow 2222/tcp
```

② Activer (ou désactiver) le pare-feu :

```
sudo ufw enable
```

```
# Pour le désactiver
```

```
sudo ufw disable
```

Règles avancées et Suppression

Pour sécuriser davantage, on peut restreindre l'accès à une adresse IP spécifique (ex : seul l'IP de l'administrateur peut accéder au SSH).

Autoriser une IP spécifique sur un port précis :

```
sudo ufw allow from 192.168.1.50 to any port 22
```

Supprimer une règle existante : Il est souvent plus simple d'afficher les règles sous forme de liste numérotée pour les supprimer.

```
# Afficher avec les numéros
```

```
sudo ufw status numbered
```

```
# Supprimer la règle numéro 3
```

```
sudo ufw delete 3
```


Filtrage via les Profils Applicatifs

Au lieu de mémoriser et d'ouvrir des numéros de ports isolés, la méthode standard sous Ubuntu consiste à utiliser l'intégration d'UFW avec les applications installées (les profils sont stockés dans le répertoire « /etc/ufw/applications.d/ »).

❶ Lister les applications reconnues par UFW :

```
sudo ufw app list
```

❷ Autoriser un service par son nom de profil :

```
# Autorise le SSH standard
sudo ufw allow OpenSSH

# Autorise le serveur web Apache (ports 80 et 443)
sudo ufw allow 'Apache Full'
```

C'est une pratique plus lisible et moins sujette aux erreurs lors des audits de sécurité de vos serveurs.

Ce qu'il faut retenir :

- **Netplan (YAML)** est le standard actuel pour déclarer la configuration réseau sous Ubuntu.
- **Backends différents** : NetworkManager (et nmcli) pour le Desktop, systemd-networkd pour le Server.
- **Diagnostic moderne** : La suite iproute2 (ip a, ip r) remplace les anciens outils (ifconfig).
- **Sécurité intégrée** : L'administration réseau va de pair avec la sécurité (**SSH** pour l'accès, **UFW** pour le filtrage).

Documentation officielle :

- **Documentation Netplan** : <https://netplan.io/>
La référence absolue pour la syntaxe YAML et les exemples complexes.
- **Ubuntu Server Guide** : <https://ubuntu.com/server/docs>
Sections dédiées au réseau, à OpenSSH et au pare-feu.

Manuels et utilitaires (En ligne de commande) :

- Les pages de manuel Linux (man) restent votre meilleur allié :
 - `man ip` (pour la suite iproute2)
 - `man nmcli` (pour NetworkManager)
 - `man sshd_config` (pour le serveur SSH)
 - `man ufw` (pour le pare-feu)